Test Project Outline – Module B – REST API Client Frontend

Competition time

Competitors will have **3 hours** to complete Module B.

Introduction

Shanghai Pudong International Airport (PVG) operates **ReClaim**, a single lost-property service for the whole airport. Passengers use ReClaim to report items they have lost, follow the progress of their claims, withdraw claims when an item is found elsewhere, and maintain their contact details.

In this module, you must build the **ReClaim passenger portal**, a consumer-facing frontend for the provided ReClaim REST API. You are responsible for the client-side user interface and user experience only. The staff desk and administration interface are outside the scope of this module.

General Description of Project and Tasks

You will be given a working solution of the Module A ReClaim API. You must use the provided solution and must not implement, replace, or modify the backend. The API documentation supplied with the solution is the source of truth for request fields, response structures, validation rules, status codes, and business rules.

Assessment aid in the provided API: `GET /passenger/claims?status=rejected` is intentionally delayed by about **2 seconds**. Other status filters respond normally. This delay exists only so out-of-order responses can be tested reliably. Do not special-case only `rejected` in your client; handle stale responses for every filter change.

Create the application as a **Single Page Application (SPA)** using a modern JavaScript framework. Additional libraries may be used. Routing must be managed by the framework, and reloading a route must restore the same page, except for unsaved form input and temporary messages.

Your API base URL: `https://module-a-solution-wsYY-YYYY.foredu.cn/api`

All API paths in this document are relative to this base URL.

The OpenAPI documentation of the backend API is available in `assets/api-docs/` — open `assets/api-docs/index.html` in a browser.

A runnable copy of the backend is also provided: unzip `assets/solution-api.zip`, import `assets/database/reclaim-db.sql` into MySQL, set the database credentials in `api/.env`, and serve it (see `RUN.md` inside the zip). It behaves identically to the hosted API, including the intentional `rejected` filter delay.

An extra helper endpoint is available: **`POST /reset-db`**. You may call it to reset the database to the canonical seed data when needed during development or testing. It is not part of the passenger portal UI you must build.

Only the following API operations are in scope:

Area	API operations
Passenger authentication	`POST /passenger/register`, `POST /passenger/login`, `POST /passenger/logout`
Passenger claims	`GET /passenger/claims`, `GET /passenger/claims/{id}`, `POST /passenger/claims`, `POST /passenger/claims/{id}/withdraw`
Passenger profile	`GET /passenger/profile`, `PUT /passenger/profile`
Public tracking	`GET /claims/track/{referenceCode}`
Terminal selection	`GET /terminals`

Do not call or create interfaces for staff authentication, terminal management, found items, staff claim processing, matching or resolution, dashboards, or `GET /my-terminals`. `GET /terminals` may only be used as a read-only source for the terminal selector in the new-claim form.

The following passenger accounts are available for testing:

Email	Password	Notes
`passenger1@email.com`	`passenger123`	Active passenger
`passenger2@email.com`	`passenger123`	Active passenger
`passenger31@email.com`	`passenger123`	Deactivated account; login must fail

Requirements

Authentication

The portal must use the passenger authentication system. An authenticated request must include the passenger token as:

```
Authorization: Bearer {token}
```

The session must survive page reloads. On sign-out, or when a request that requires authentication receives a `401 Unauthenticated` response, the current session must be cleared. Only passenger credentials and tokens may be used; credentials and tokens of staff or system operators with elevated privileges must not.

Application shell and navigation

The application must have a clear, consistent passenger-facing layout and navigation. Visual design polish is not the most important factor; a working, usable application that correctly supports the required passenger workflows is. A Font Awesome package is provided in `assets/fontawesome` for icons if you choose to use it.

Use **path-based** client routes (framework History mode). These paths are mandatory for assessment — markers may open them directly. Do not use hash routing (`#/...`).

- Unauthenticated visitors must be able to register, sign in, and use public claim tracking.

- Authenticated passengers must additionally be able to file a claim, view their claims, open a claim's details, edit their profile, and sign out.
- Authenticated routes must be guarded. If an unauthenticated visitor requests one, navigate them to the sign-in page and return them to the originally requested URL after successful authentication.
- The default route `/` must redirect authenticated passengers to the My claims page and others to the sign-in page.
- Controls and navigation for staff or administrative functions must not appear.
- The interface must make these situations visually and textually distinct so the passenger always understands what is happening:
    - **Loading** — data or an action is in progress (for example a spinner, skeleton, or "Loading…" message). Do not show an empty list or a success message while waiting.
    - **Empty** — the request succeeded but there is nothing to show (for example "You have no claims yet"). This must not look like an error or like loading.
    - **Success** — an action completed correctly (for example a confirmation after filing a claim or updating the profile).
    - **Error** — a request failed or validation prevented submission; show a clear message and, where useful, a way to retry.

Registration page

The registration page route is `/register`. It must create a passenger account.

The form must collect:

- First name
- Last name
- Email address
- Phone number (optional)
- Address line 1
- Address line 2 (optional)
- City
- Postcode
- Country
- Password, with a minimum length of 8 characters

Validate required fields, email format, and password length before submission. Display field-specific validation messages returned by the API. A successful registration returns a passenger token and passenger details; store the session and navigate to the My claims page.

Sign-in page and sign out

The sign-in page route is `/login`. It must accept an email address and password and authenticate through the appropriate endpoint.

- On success, store the returned passenger token and passenger details and continue to the originally requested authenticated URL, or to the My claims page if none was requested.
- Invalid credentials or an inactive account return `401`; show a clear message without exposing sensitive information.
- An authenticated passenger must be able to sign out from the application shell.

- Signing out must clear the local session and navigate to the sign-in page. The local session must still be cleared if the remote token is already invalid.

New claim page

The new-claim page route is `/claims/new`. Authenticated passengers must be able to report a lost item here.

Implement claim reporting as an **assisted multi-step process**:

1. **Where and when** — select the terminal and date lost.
2. **Item details** — enter the category, brand, colour, description, and flight number.
3. **Review and submit** — show a complete summary and allow the passenger to return to either previous step without losing data.

The journey must show the current step and completed steps. Passengers may move forward only when the current step is valid. Browser back and forward navigation must not accidentally submit the form or lose already entered values.

The form must contain:

Field	Requirement
Terminal	Required
Category	Required. Use exactly: `electronics`, `documents`, `luggage`, `clothing`, `jewellery`, `keys`, `other`.
Brand	Optional
Colour	Optional
Description	Required
Date lost	Required. Submit as `lost_on`; a future date must not be accepted.
Flight number	Optional

Client-side validation must prevent clearly invalid submissions. API validation errors must be shown beside the relevant fields where possible.

An unfinished claim draft must be recoverable after a page reload or when the browser is reopened: offer to continue it or discard it. A previous draft must not be retrieved if it was already submitted or discarded, or if a different passenger is signed in.

While terminals are loading, show a loading state and do not allow the passenger to continue without a selected terminal. If loading fails, show an error and a retry action. The passenger must not submit a claim without selecting a terminal from a successful terminals response.

Client-side claim quality score

To help passengers provide useful identifying information, calculate a **claim quality score** from 0 to 100 entirely in the browser. This is a client-side guidance feature: it must not call an API, must not be sent with the

claim, and must not be presented as a probability that the item will be recovered.

Recalculate the score immediately whenever a relevant form value changes:

Rule	Points
A terminal is selected	10
A category is selected	10
Brand contains a non-whitespace value	15
Colour contains a non-whitespace value	10
Flight number contains a non-whitespace value	10
Character threshold 1: Trimmed description contains at least 40 characters	10
Character threshold 2: Trimmed description contains at least 80 characters	+10
Description contains at least 12 distinct words, compared case-insensitively and ignoring punctuation	10
Date rule 1: `lost_on` is today or 1 day ago	15
Date rule 2: `lost_on` is 2–3 days ago	10
Date rule 3: `lost_on` is 4–7 days ago	5
Date rule 4: `lost_on` is 8 or more days ago, missing, invalid, or in the future	0

Only one date rule may contribute to the score. Character thresholds are cumulative, so a description of at least 80 characters receives 20 description-length points. Clamp the final result to the range 0–100.

Display both the numerical score and its label:

Score	Label
0–39	Needs more detail
40–69	Basic
70–89	Detailed
90–100	Excellent

The review step must show a points breakdown and indicate which optional details could still increase the score. For example, a claim with a terminal and category, brand, colour, no flight number, an 85-character description containing at least 12 distinct words, and today's date scores **90**.

The score is advisory and must never prevent submission when the API-required fields are valid. When a draft is continued or discarded, the score must match the form values again.

After successful creation:

- Show a **confirmation modal** with the newly created claim details from the API response. At least show the reference code (`CL-XXXXXXXX`), initial `submitted` status, category, terminal, and date lost. Make the

reference code prominent and easy to copy.
- The modal must offer exactly these two actions:
  - **Create a new claim** — close the modal, reset the multi-step form to the first step, and leave the passenger ready to file another claim.
  - **Show my claims** — navigate to the My claims page.
- If the passenger dismisses the modal in any other way (Escape, a close button, or clicking the backdrop), treat that the same as **Show my claims**: navigate to the My claims page. Do not return them to the finished form with no confirmation left on screen.

My claims page

The My claims page route is `/claims`. It must retrieve only the authenticated passenger's claims.

- Display claims newest first as returned by the API.
- Support the API's paginated response using its `links` or `meta` information. Do not assume that all claims are returned on one page.
- Show at least the reference code, category, terminal, date lost, and current status for each claim.
- Allow filtering by status.
- Each result must link to its claim detail page.

The claims page must behave as a **stateful claims workspace**. Store the selected status and page number in the URL query string (for example `?status=submitted&page=2`). Reloading the page, sharing its URL, and using browser back or forward must restore the same view. Changing the status must return to page 1.

Requests can finish in a different order when a passenger changes filters quickly. Cancel obsolete requests when possible, or otherwise ensure that an older response can never overwrite the latest selected view. Use the provided API's intentional delay on `?status=rejected` to verify this: select **rejected**, then quickly switch to another status (for example **resolved**) before the rejected response returns. After both requests have finished, the list and URL must still show the last selected status — not rejected.

Keep the existing results visible with a non-blocking loading indication while moving between pages; do not briefly show an incorrect empty state.

Claim detail and withdrawal page

The claim detail page route is `/claims/:id`. Retrieve a claim and present all useful fields returned by the passenger claim response, including:

- Reference code
- Terminal
- Category
- Brand and colour when available
- Description
- Date lost
- Flight number when available
- Status
- Created and updated dates
- Resolution date when available
- Matched-item information when the API includes it

A passenger may withdraw their own claim only while its status is `submitted` or `under-review`.

- Show the withdrawal action only when the current status is eligible.
- Ask for confirmation before sending the request.
- On success, update the displayed claim to the returned `withdrawn` state.

Live claim journey

Present the claim's current state as a passenger-friendly journey rather than only displaying the raw status value. The journey must correctly represent the API statuses `submitted`, `under-review`, `matched`, `resolved`, `rejected`, and `withdrawn`. Rejected and withdrawn claims are alternative endings and must not be presented as successfully completed journeys.

Claim refreshing

While an authenticated claim detail page is open, refresh it every **15 seconds**:

- Do not start a new refresh while the previous refresh is still running.
- Pause automatic refresh while the page is not visible and refresh immediately when it becomes visible again.
- When the status changes, update the journey and claim information without a full page reload and announce the change visibly and through an accessible live region.
- Stop polling when the claim reaches `resolved`, `rejected`, or `withdrawn`.
- Provide a **manual refresh** control. It must use the same loading and error handling as automatic refresh.
- A temporary refresh failure must keep the last successful claim visible and show a non-destructive warning. Do not replace the claim with an empty or full-page error state. The manual refresh control may be used to retry.

Public claim tracking page

The public claim tracking entry route is `/track`, and a tracking result uses `/track/:referenceCode`. Anyone must be able to track a claim without signing in by entering its reference code.

- Accept a reference code in the `CL-XXXXXXXX` format.
- On success, navigate to the tracking result route and show only the public response data: reference code, status, category, terminal, date lost, creation date, and resolution date when available.
- Do not attempt to retrieve or display passenger personal data.
- A `404` response must produce a clear "claim not found" result and allow another search.

The reference code must appear in the tracking result route so that it can be bookmarked or shared. Reloading that URL must repeat the public lookup. Normalize user input to uppercase and remove accidental surrounding whitespace before requesting it, but do not change the internal characters.

Apply the same live journey and 15-second refresh behaviour as the authenticated claim detail page. Public tracking must never call an authenticated passenger endpoint or reveal fields outside the public response.

Passenger profile page

The profile page route is `/profile`. It must retrieve the authenticated passenger's profile details and allow updates.

Display and allow editing of:

- First name
- Last name
- Email address
- Phone number
- Address line 1
- Address line 2
- City
- Postcode
- Country

The passenger may also set a new password of at least 8 characters. Do not display an existing password and do not send an unchanged or empty password field.

The update endpoint supports partial updates. Show validation errors, including a non-unique email address, next to the relevant fields. After a successful update, show the returned current profile data and a visible success confirmation.

Error handling

Handle at least the following error cases throughout the application:

Status or condition	Required behaviour
`401 Unauthenticated` or invalid credentials	For a protected request, clear the invalid session and request sign-in. For the login form, show an authentication error.
`404 Not Found`	Explain that the claim or resource was not found or is unavailable to this passenger.
`422 Validation failed`	Display the general message and field-specific errors.
`422` business-rule failure	Display the API's descriptive message near the action that failed.
Network or unavailable API	Explain that the service could not be reached and allow the user to retry.

Prevent duplicate submissions while a request is in progress. Do not silently discard API errors.

The application must prevent stale responses from updating a page after its route or authenticated passenger has changed. Automatic and manual refreshes, route changes, and sign-out must not leave timers or pending requests applying to the wrong page. Timers and pending requests must be cleaned up when their page is left.

Design, responsiveness, and accessibility

The result must look and behave like a public airport passenger service, not an internal administration dashboard.

- Use a clear visual hierarchy and readable status indicators. Status must not be communicated by colour alone.
- Make forms usable with keyboard navigation and associate labels and error messages with their controls.
- Provide visible focus states and meaningful text for interactive controls.
- Confirm destructive or consequential actions such as claim withdrawal.
- The application must remain usable without horizontal scrolling at common mobile and desktop viewport widths.
- Use appropriate date, loading, and feedback presentation consistently across the portal.

Assessment

Module B will be assessed in the provided latest stable version of Google Chrome. Assessment will include:

- Correct integration with the provided Module A API
- Completeness and correctness of passenger workflows
- SPA routing, authentication persistence, and route protection
- Multi-step form state, passenger-scoped draft recovery, and validation
- Correct client-side claim quality calculation, breakdown, and guidance
- URL-driven pagination and filtering, including correct handling of competing requests (verified with the delayed `rejected` filter)
- Live claim journeys, polling lifecycle, and status-change feedback
- Error handling and recovery from unavailable or stale API requests
- User experience, responsive behaviour, and accessibility
- Appropriate framework use and maintainable frontend code

Any modification to the provided backend, its database, or its endpoint contract will not be considered during assessment. Only the documented consumer-facing operations listed in this task may be used.

Mark distribution

WSOS SECTION	Description	Points
1	Work organization and self-management	4
2	Communication and interpersonal skills	3
3	Design Implementation	8.5
4	Front-End Development	17.5
5	Back-End Development	0
Total		**33**